Payanam Bus Booking — Technical Document

Oct 13, 2026 · @Anitha

Overview

Payanam is a bus ticket booking website for four Tamil Nadu cities: Chennai, Coimbatore, Erode and Tirupur. Travellers search a route, pick a date from a calendar that blocks past dates, choose seats on a seat map, enter passenger details, go through a demo payment step and receive a ticket with a booking ID.

The project exists in two versions that share the same front end and booking rules:

Version	Where it runs	How bookings are stored	Use
Node.js app	Any host that runs Node.js 18+	`data/bookings.json` on the server	Code for the Firstwebsitecreation GitHub repo
Hosted page	claude.ai artifact	The page's shared database	Live demo you can open and share

All operators, departure times, durations and fares are **sample data** and are labelled as such on the site. The payment step is a demo: no money is charged and no card or UPI details are collected.

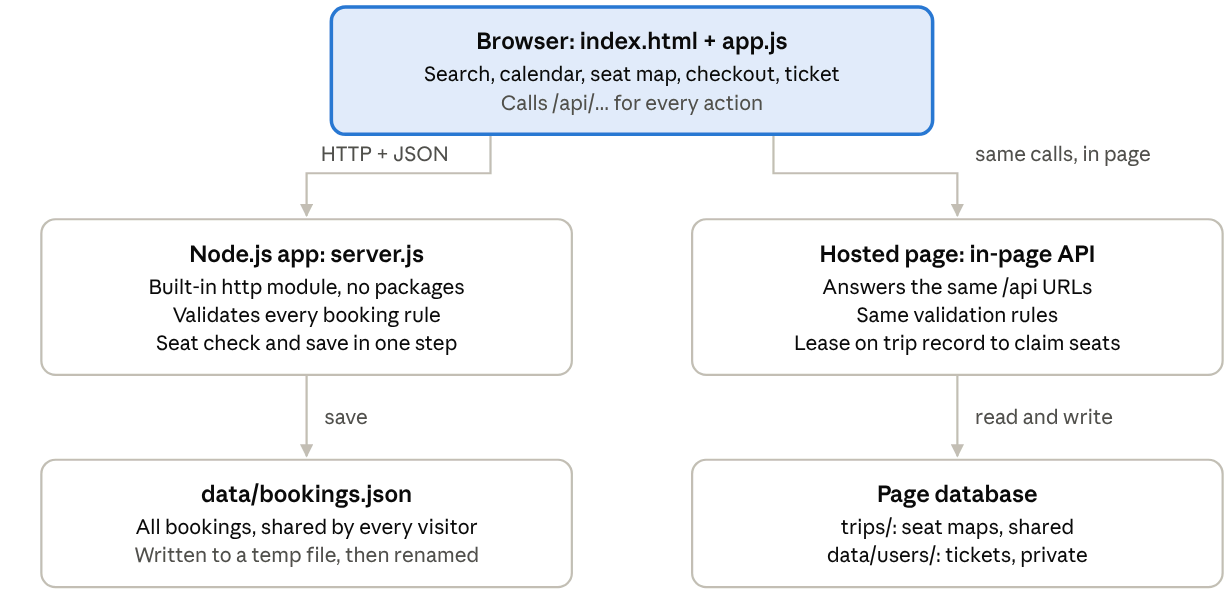
Features

Feature	What it does
Route search	Any of the 12 directions between the four cities; 6 buses per direction, 72 in total
Calendar date picker	Custom month calendar; past dates are crossed out and cannot be clicked; Today and Tomorrow shortcuts

Feature	What it does
Departed buses	On today's date, buses whose departure time has passed (Indian Standard Time) show "Departed" and cannot be booked
Filters and sorting	Filter by AC, Non-AC, Sleeper or Seater; sort by departure time, lowest fare or most seats left
Seat selection	2+2 seater layout (40 seats) or sleeper layout with lower and upper decks (30 berths); up to 6 seats per booking
Passenger details	Name, age and gender per seat; 10-digit mobile number; optional email
Demo payment	Choose UPI, Card or Net banking; nothing is charged
Ticket	Booking ID, route, times, seats, passengers and total fare
My bookings	Node.js app: look up by booking ID and mobile number. Hosted page: list of your own bookings
Front page	Photo banner, popular-route shortcuts, feature strip, bus illustrations on results, summary and ticket
Themes and devices	Works at phone width; the hosted page also supports dark mode

Architecture

The browser code is the same in both versions; only what answers its `/api` calls differs.



architecture · front end and the two back ends

In the Node.js app, `server.js` answers over HTTP and saves to a JSON file. In the hosted page, an in-page layer answers the same calls and stores seat maps and tickets in the page's database.

Project structure

The Node.js app has no third-party dependencies; everything runs on Node's built-in modules.

File	Purpose
`server.js`	HTTP server (Node `http` module): serves `public/`, implements the booking API, validates every request, reads and writes bookings
`data/buses.js`	Sample cities, routes, operators, bus types, schedules, fare rates and seat layouts
`data/bookings.json`	Created automatically on the first booking; holds all bookings; ignored by git

File	Purpose
`public/index.html`	Page markup: header, photo banner, search form, results, checkout, ticket, booking lookup, home sections
`public/styles.css`	All styling, including responsive rules and bus illustration colours
`public/app.js`	Front-end logic: calendar, search, filters, seat map, checkout, ticket, popular routes, bus illustrations
`public/images/bus-hero.jpg`	Front-page banner photo (840 × 495 px)
`package.json`	Project metadata and the `npm start` script
`README.md`	Setup, structure, data editing, API summary, hosting notes
`.gitignore`	Excludes `node_modules/` and booking data

The hosted page is a single HTML file that bundles the same markup, styles and front-end code, plus a copy of the sample data and an in-page version of the API.

Data model

Buses are generated at start-up from a few small tables in `data/buses.js`, so changing a table changes every route that uses it.

Routes (sample distance and travel time, used in both directions):

Route	Distance (km)	Travel time
Chennai – Coimbatore	505	9h 00m
Chennai – Tirupur	455	8h 20m
Chennai – Erode	400	7h 30m
Coimbatore – Erode	100	2h 00m
Erode – Tirupur	55	1h 20m
Coimbatore – Tirupur	50	1h 15m

Bus types — fare = distance × rate, rounded to the nearest ₹10, minimum ₹60:

Bus type	Rate (₹/km)	Layout	Seats
AC Sleeper	2.4	Sleeper, lower + upper deck	30
Non-AC Sleeper	1.8	Sleeper, lower + upper deck	30
AC Seater (2+2)	1.7	Seater	40
Non-AC Seater (2+2)	1.3	Seater	40

Schedules. Routes over 300 km use a mostly overnight schedule (20:30, 21:15, 21:45, 22:30, 23:00 and 07:00). Shorter routes run through the day (06:00, 08:30, 11:00, 14:00, 17:30, 20:00). Six sample operators are assigned in rotation.

Bus record: `id` (e.g. `CHE-CBE-01`), `operator`, `type`, `layout`, `from`, `to`, `depart`, `arrive`, `nextDay`, `durationMins`, `distanceKm`, `fare`.

Seat IDs. Seaters use rows 1–10 and columns A–D (`1A` … `10D`) with the aisle between B and C. Sleepers use `L1` – `L15` (lower) and `U1` – `U15` (upper) in a 1 + aisle + 2 layout.

Booking record: `id` (`PYN` + 8 hex characters), `status`, `busId`, `operator`, `busType`, `from`, `to`, `date`, `depart`, `arrive`, `nextDay`, `seats[]`, `passengers[]` (`seat`, `name`, `age`, `gender`), `contact` (`phone`, `email`), `farePerSeat`, `totalFare`, `payment` (`method`, `demo: true`), `createdAt`.

Business rules and validation

Every rule is checked twice: in the browser for quick feedback, and again on the server (or the hosted page's booking layer) so it cannot be bypassed. All dates and times use Indian Standard Time (`Asia/Kolkata`), whatever the visitor's own time zone.

Rule	Detail	Message shown
No past dates	Travel date must be a real `YYYY-MM-DD` date on or after today in IST; past days are disabled in the calendar	Tickets cannot be booked for a past date. Please choose today or a future date.

Rule	Detail	Message shown
No departed buses	On today's date, a bus whose departure time is at or before the current IST time cannot be booked	This bus has already departed today. Please choose another bus or date.
Different cities	From and To must be valid cities and must differ	From and To cities must be different.
Seat limit	1 to 6 seats per booking; each seat must exist on that bus and appear once	You can book up to 6 seats at a time.
Passenger name	2–60 characters after trimming	Each passenger needs a name (2–60 characters).
Passenger age	Whole number from 1 to 120	Please enter a valid age for …
Gender	Male, Female or Other	Please choose a gender for …
Mobile number	10 digits starting with 6, 7, 8 or 9	Please enter a valid 10-digit mobile number.
Email	Optional; must look like an email if given	Please enter a valid email address or leave it blank.
Payment method	UPI, Card or Net Banking	Please choose a payment method.
No double booking	Seats are re-checked immediately before saving; if any is taken the booking is refused and the visitor picks again	Sorry, seat … was just booked by someone else. Please pick again.

API reference

The Node.js server exposes five JSON endpoints. The hosted page answers the same URLs inside the page, so the front-end code is identical in both versions.

Method	Path	Input	Success response
GET	`/api/meta`	—	`{ cities, today, maxSeats }`

Method	Path	Input	Success response
GET	`/api/buses`	Query: `from`, `to`, `date` (YYYY-MM-DD)	`{ date, buses: [bus + totalSeats, seatsAvailable, departed] }`, sorted by departure
GET	`/api/buses/:id/seats`	Query: `date`	`{ bus, seats: [{ id, deck, row, col }], booked: [seatId], departed }`
POST	`/api/bookings`	JSON: `busId`, `date`, `passengers[]`, `contact { phone, email }`, `paymentMethod`	`201` + `{ booking }`
GET	`/api/bookings/:id`	Query: `phone`	`{ booking }` (Node.js app only)

Example booking request:

```
{
  "busId": "CHE-CBE-01",
  "date": "2026-10-14",
  "passengers": [{ "seat": "L1", "name": "Anitha K", "age": 34, "gender":
"Female" }],
  "contact": { "phone": "9876543210", "email": "" },
  "paymentMethod": "UPI"
}
```

Errors return `{ "error": "<message>" }` with these status codes:

Status	When
400	Invalid input, past date, departed bus, bad passenger or contact details
403	Hosted page only: the viewer has view-only access
404	Unknown bus, booking or route

Status	When
409	One or more seats were taken first; the body also lists them in `taken`
500	Unexpected server error
503	Hosted page only: storage unreachable or another booking on the same bus is in progress

Storage and concurrency

Both versions make a booked seat visible to every visitor and refuse a second booking of the same seat.

Node.js app. Bookings are held in memory and saved to `data/bookings.json` after each booking. Each save writes a temporary file and renames it over the old one, so a crash mid-write cannot leave a half-written file. Node handles one request at a time on its main thread, so the seat check and the save run without interruption, which prevents double booking. Booked seats for a bus and date are worked out from confirmed bookings.

Hosted page. Data is split so that personal details stay private:

Location	Contents	Who can read it
`trips/<busId>_<date>`	Map of seat ID → booking ID; no names or phone numbers	Everyone who can open the page
`data/users/<viewer id>/<bookingId>`	The full ticket, including passenger names and mobile number	Only the person who booked

To claim seats, the page takes a 5-second lease on the trip document, re-reads it, refuses if any chosen seat is taken, then writes the new seat map. If another visitor holds the lease, it retries up to 5 times, 0.7 s apart. An open seat map subscribes to its trip document, so seats booked by others turn grey live and any of them the viewer had selected are removed with a message. Viewers with view-only access can browse but not book. If the database is unavailable (for example, signed out), bookings are kept only in that browser tab and the banner says so.

Front end

The site is one page with plain HTML, CSS and JavaScript and no framework. Sections are shown and hidden as the visitor moves through the booking.

1. **Home** — photo banner with the headline and a "பயணம் · Payanam" brand line, search form overlapping the banner, six popular-route cards and a four-point feature strip.
2. **Results** — bus cards with a bus illustration, operator, type, departure and arrival (with +1 for next-day arrival), fare and seats left; type filter and sort control.
3. **Seat map** — opens inside the bus card; legend for available, selected and booked; running total and Continue button.
4. **Passenger details** — one row per seat plus contact details, beside a trip summary with a bus illustration.
5. **Payment** — demo method choice and a Pay button that shows the total.
6. **Ticket** — booking ID, confirmed status, bus illustration, trip details and passenger table.

Calendar. A custom month grid built in `app.js`, so past dates are disabled the same way in every browser. The previous-month button is disabled on the current month. Today is outlined; the selected day is filled. It closes on outside click or Escape.

Seat map. Seats are placed with CSS grid using each seat's `row` and `col`. Seaters use 5 columns (aisle in the middle); sleepers use 4 columns per deck with taller berths.

Bus illustrations. Drawn as inline SVG by `busArt(type, size)`: teal body for AC, orange for Non-AC, two rows of windows for sleepers and a roof unit for AC. They are original drawings, so no image files are needed.

Styling. Plus Jakarta Sans for text and Baloo Thambi 2 for headings (Google Fonts). Colours are CSS variables; the hosted page defines a dark palette as well. Layouts collapse to one column at 860 px and 560 px.

Accessibility. Form fields have labels, error messages use `role="alert"`, seat buttons announce their state, and focus outlines are visible. User-entered text is escaped before display.

Setup, testing and deployment

Run locally. Install Node.js 18 or newer, unzip the project, then:

```
cd bus-booking
npm start
```

Open http://localhost:3000. There is no `npm install` step. Set the `PORT` environment variable to use another port.

Tests performed (2026-10-13):

Area	Check	Result
API	Past date rejected for search and for booking	Pass
API	Invalid date (2026-02-30) and same From/To rejected	Pass
API	All 12 directions return 6 buses	Pass
API	Today's buses already gone marked as departed	Pass
API	Second booking of the same seat returns 409	Pass
Server	Paths outside `public/` (e.g. `/../server.js`) return 404	Pass
Browser	Calendar disables past days and the previous-month button	Pass
Browser	Full flow: search → seats → passengers → payment → ticket → lookup	Pass
Browser	Popular-route card opens that route's results	Pass
Browser	No horizontal scrolling at 390 px phone width; no script errors	Pass
Hosted page	Seat map written to the database is readable by a view-only viewer	Pass

Not tested: two people booking the same seat at the same moment on the hosted page.

Deployment. The Node.js app needs a host that runs Node, such as Render, Railway or a VPS. GitHub Pages cannot run it, because it only serves static files. Some free hosts erase local files on restart, which would delete `data/bookings.json`; for real use, move bookings to a database.

Limitations and next steps

Limitation	Suggested next step
All buses, times and fares are sample data	Replace the tables in `data/buses.js` with real operator schedules and fares
Payment is a demo	Integrate a payment gateway and confirm seats only after payment succeeds
Bookings stored in one JSON file	Move to a database (for example PostgreSQL or MongoDB) for durability and multiple servers
No user accounts or OTP	Add login or OTP verification of the mobile number
No cancellation	Add cancel and refund flow that frees the seats
Booking lookup needs ID + mobile only	Add rate limiting to stop guessing
Banner photo is 840 px wide and shows another company's branding (Intercity travels, BharatBenz)	Use a photo you own or have permission for, at least 1600 px wide
Node.js app and hosted page keep separate bookings	Point both at one shared backend if both stay in use